

USART

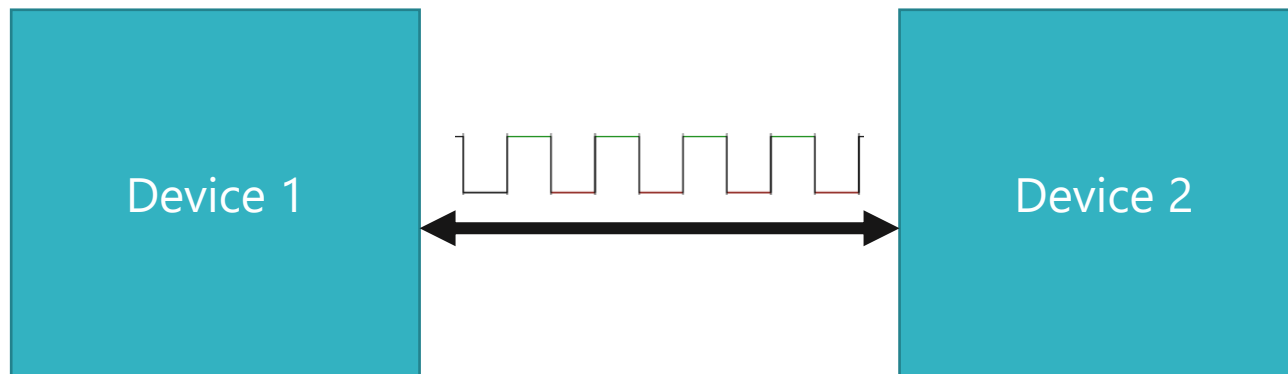
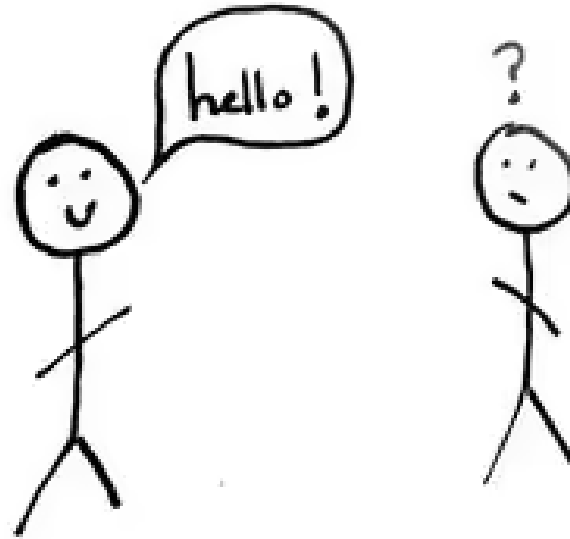
*Universal
synchronous
asynchronous
receiver
transmitter*

27 Sept 2024



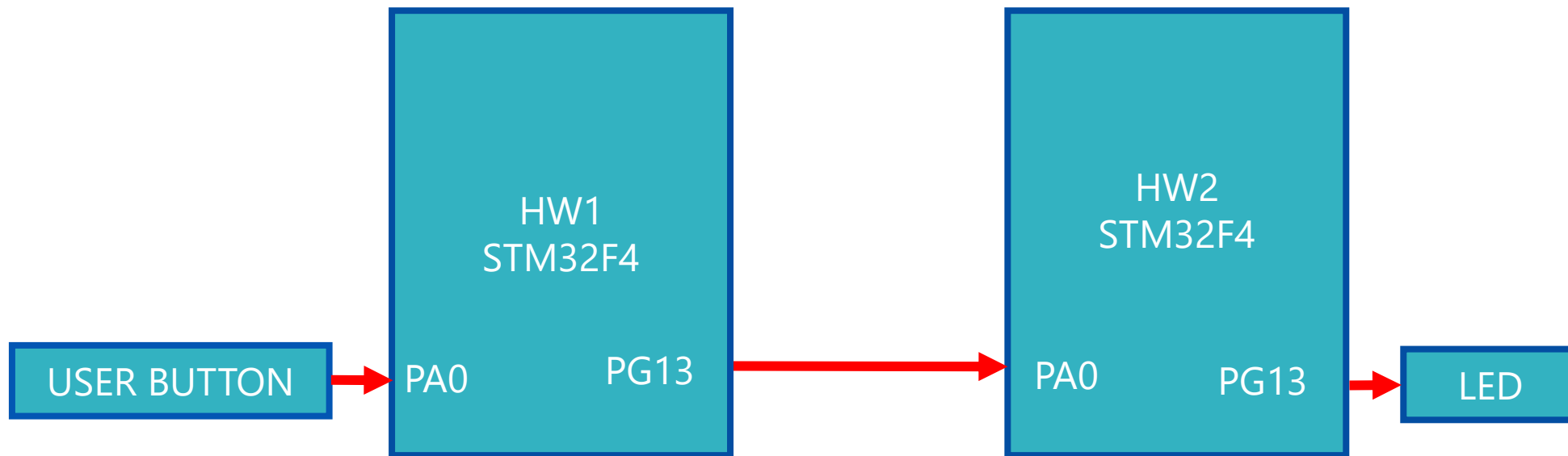
- 1. Communication overview**
- 2. UART overview**
- 3. USART**
- 4. USART Interrupt**

1. Communication overview



1. Communication overview

- Requirement:** Using user button at HW1 to change led state at HW2 via GPIO
- if user button at HW1 is pressed then led at HW2 is on
 - if user button at HW1 is released then led at HW2 is off



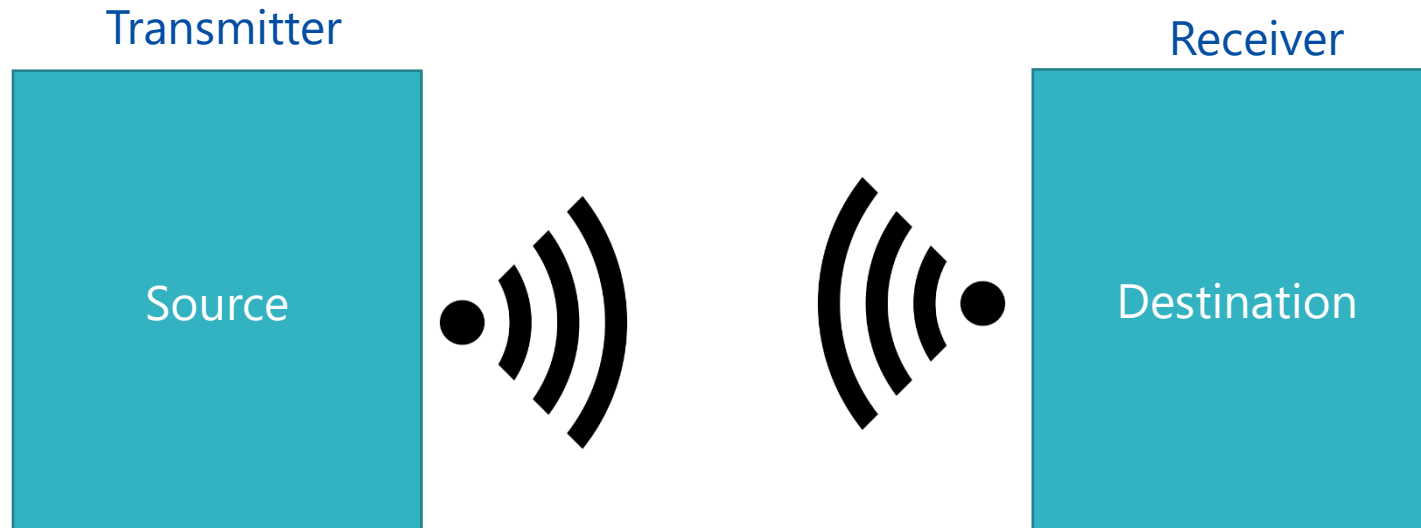
Connection Diagram

1. Communication overview

Wired

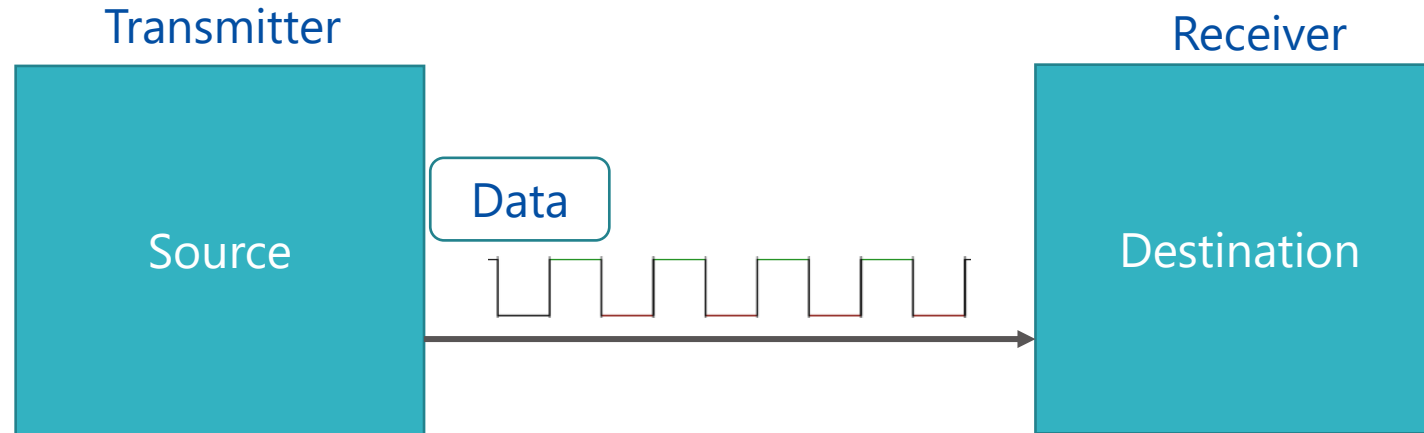


Wireless

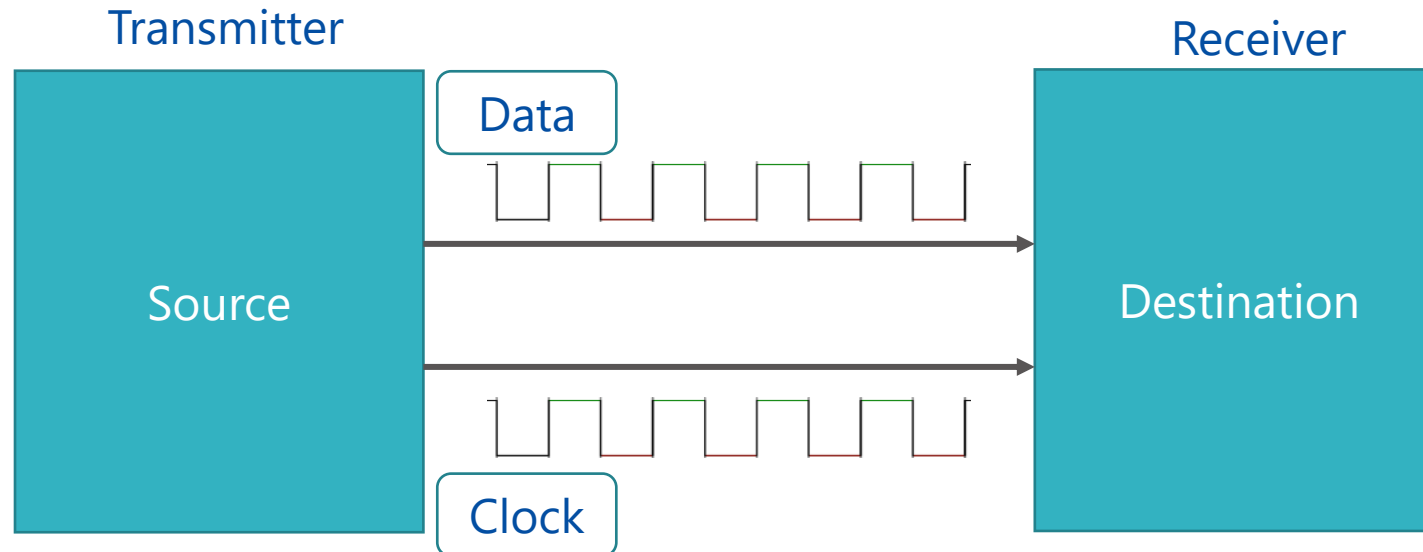


1. Communication overview

Asynchronous

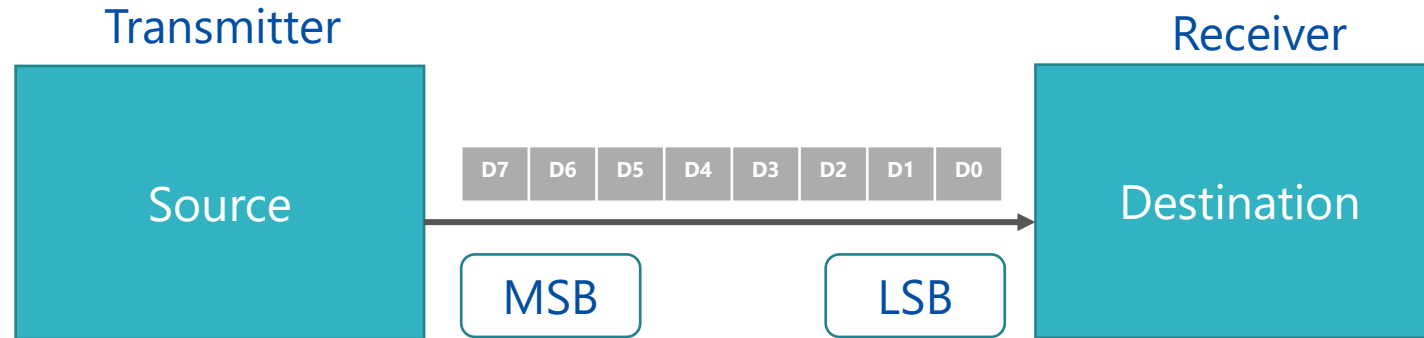


Synchronous

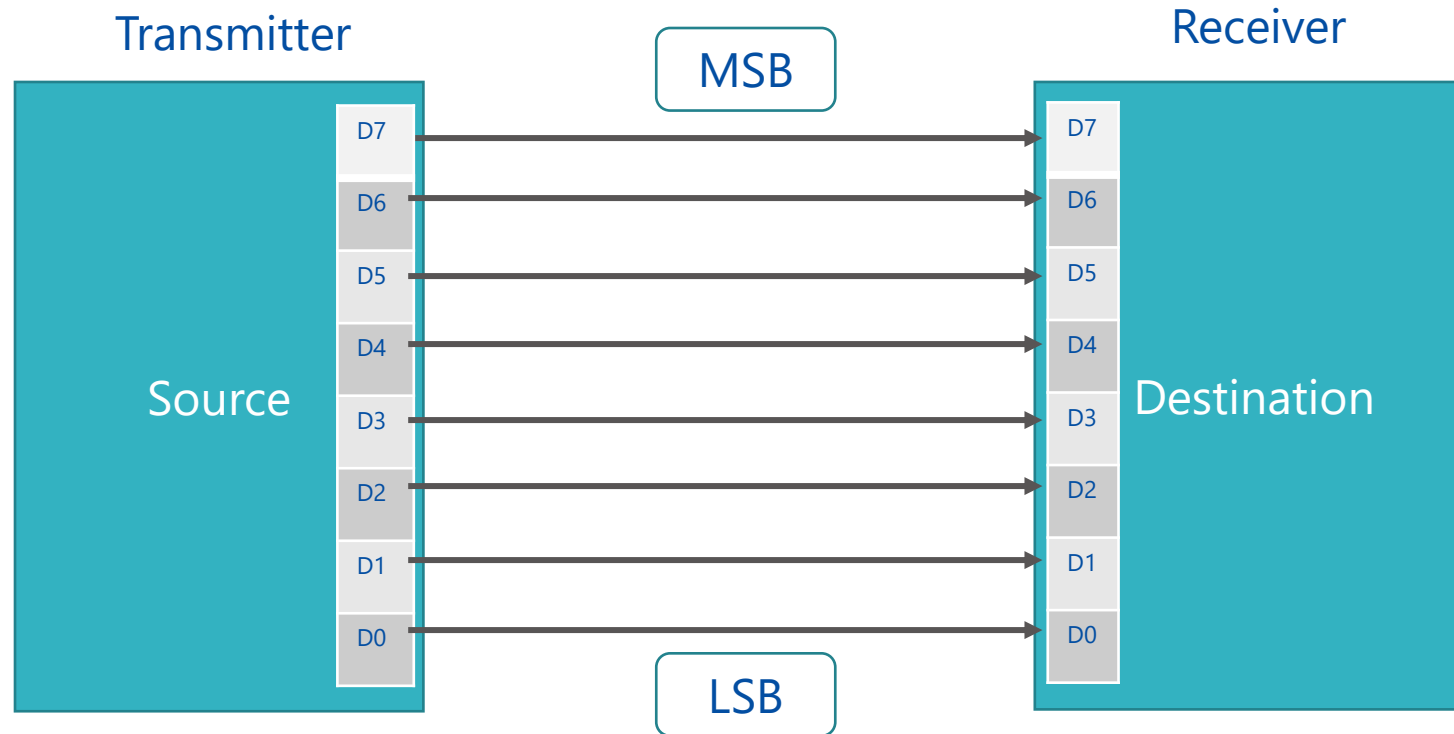


1. Communication overview

Serial

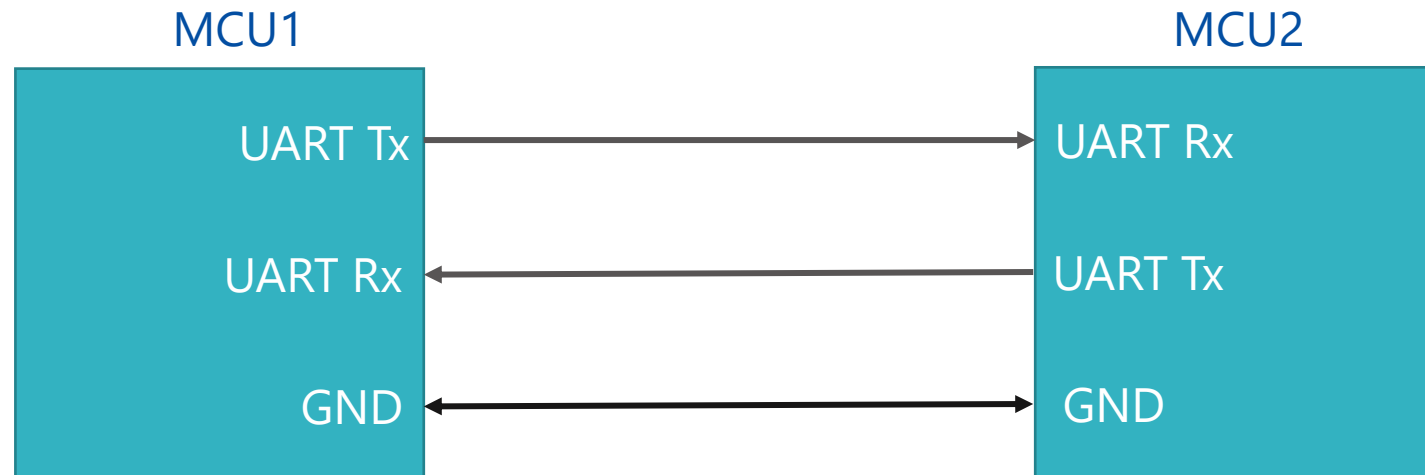


Parallel



2. UART overview

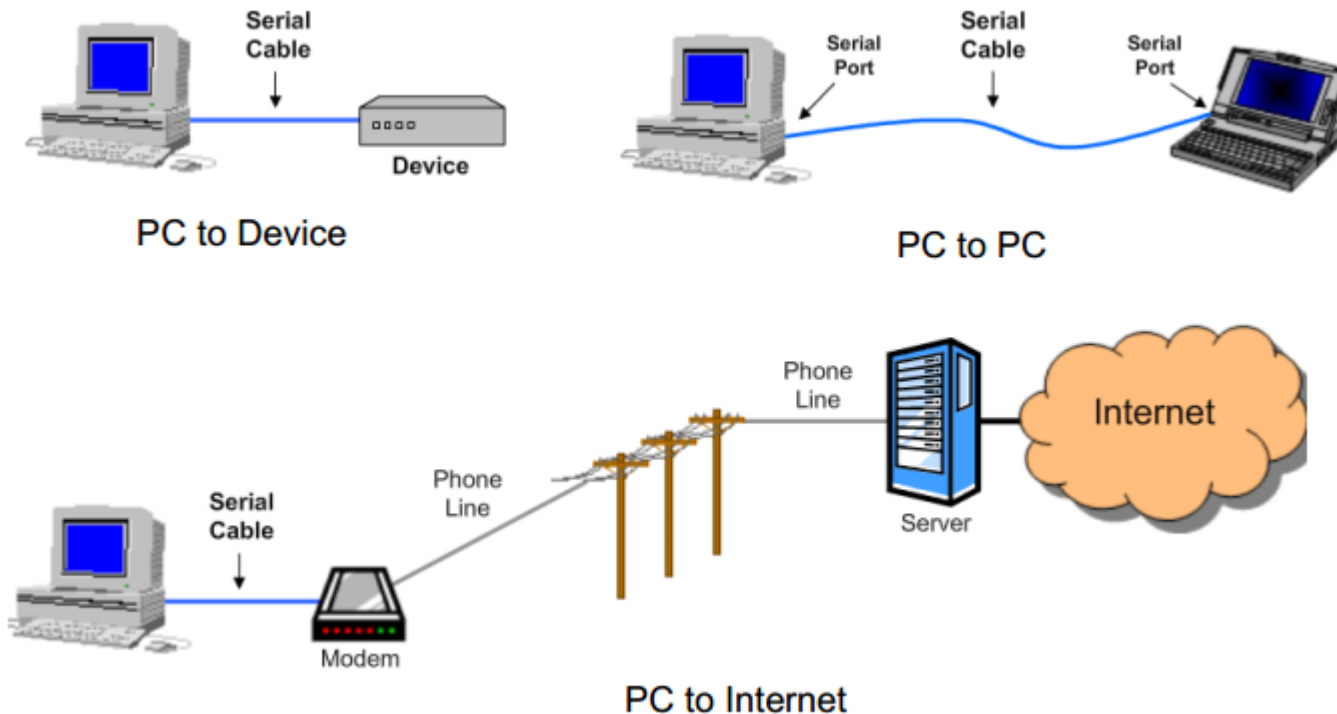
- ❖ Universal Asynchronous Receiver/ Transmitter
- ❖ Protocol for exchanging serial data between two devices
- ❖ Uses only two wired
 - Tx to Rx (each direction)
- ❖ Can be simplex, half-duplex or full-duplex
- ❖ Data is transmitted as frames



2. UART overview

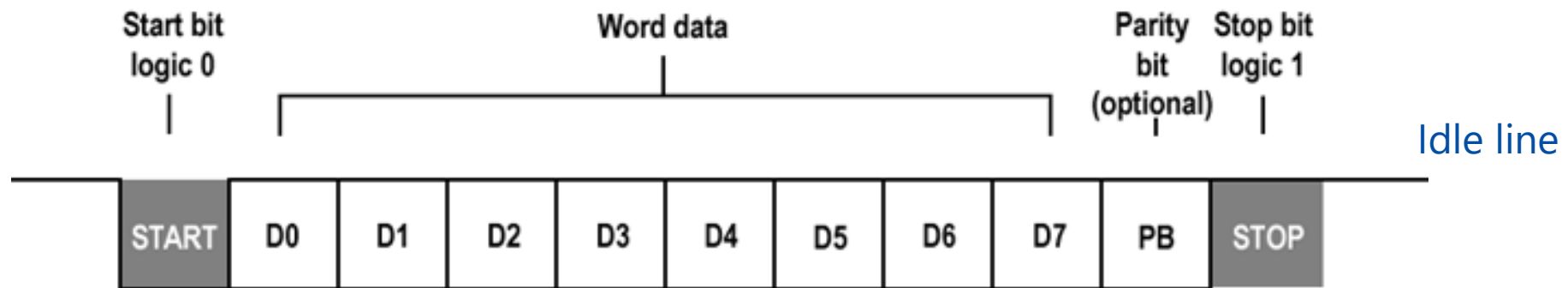
Where is UART used?

- Serial(COM) port
- RS-232
- RS-485
- Communicate with MCU, sensors...
- LIN, CxPI



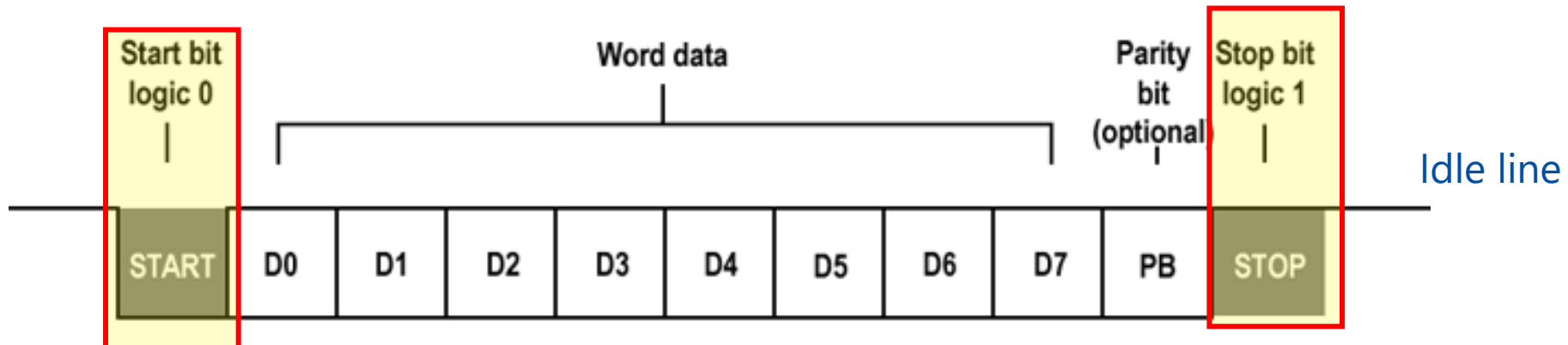
2.1. UART frame format

- ❖ **UART frames consist of:**
 - **Start/Stop bits**
 - **Data bits**
 - **Parity bit(optional)**
- ❖ **High voltage(mark=1), low voltage(space=0)**
- ❖ **In the idle state, the line is held high**



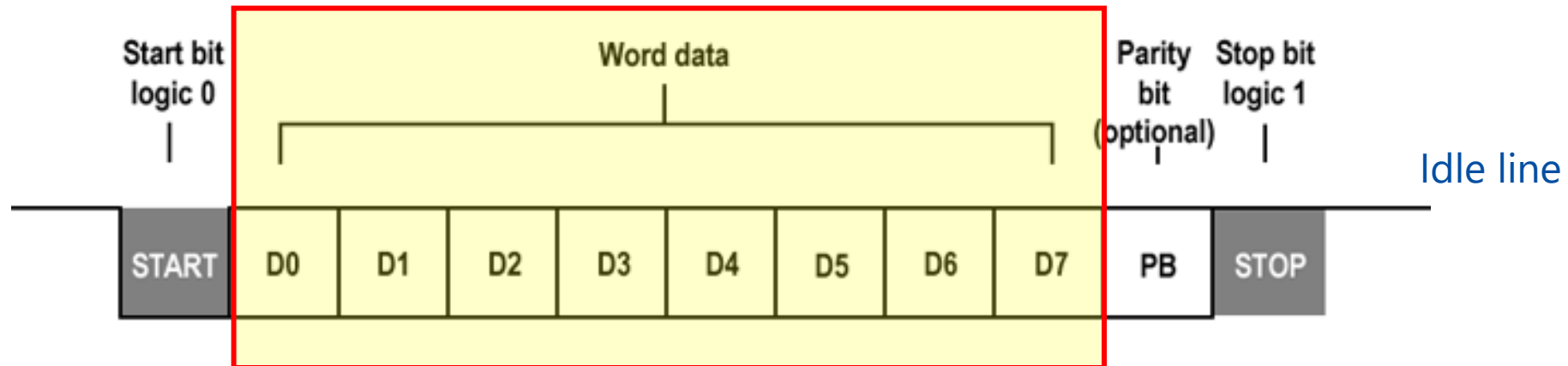
2.2. Start and stop bits

- ❖ The start bit indicates data is coming
 - Transition from idle (high) to low
- ❖ The stop bit(s) indicate data is complete
 - Stay/ Return to idle(high)
 - Second(optional) stop bit



2.3. Data bits

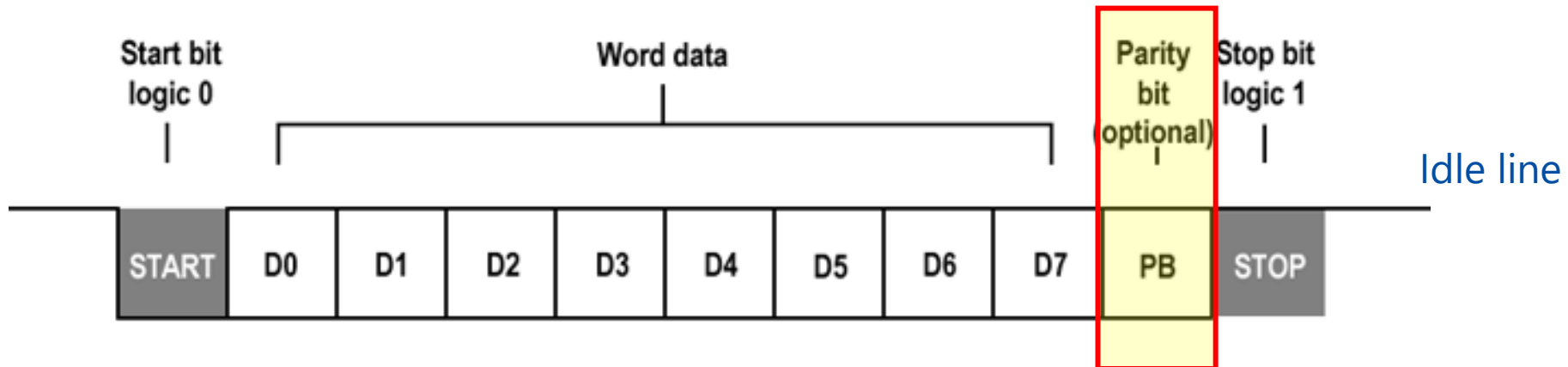
- ❖ User data
- ❖ Length: 5 to 9 (usually 7 or 8)
- ❖ Data is typically sent with the least significant bit (LSB)



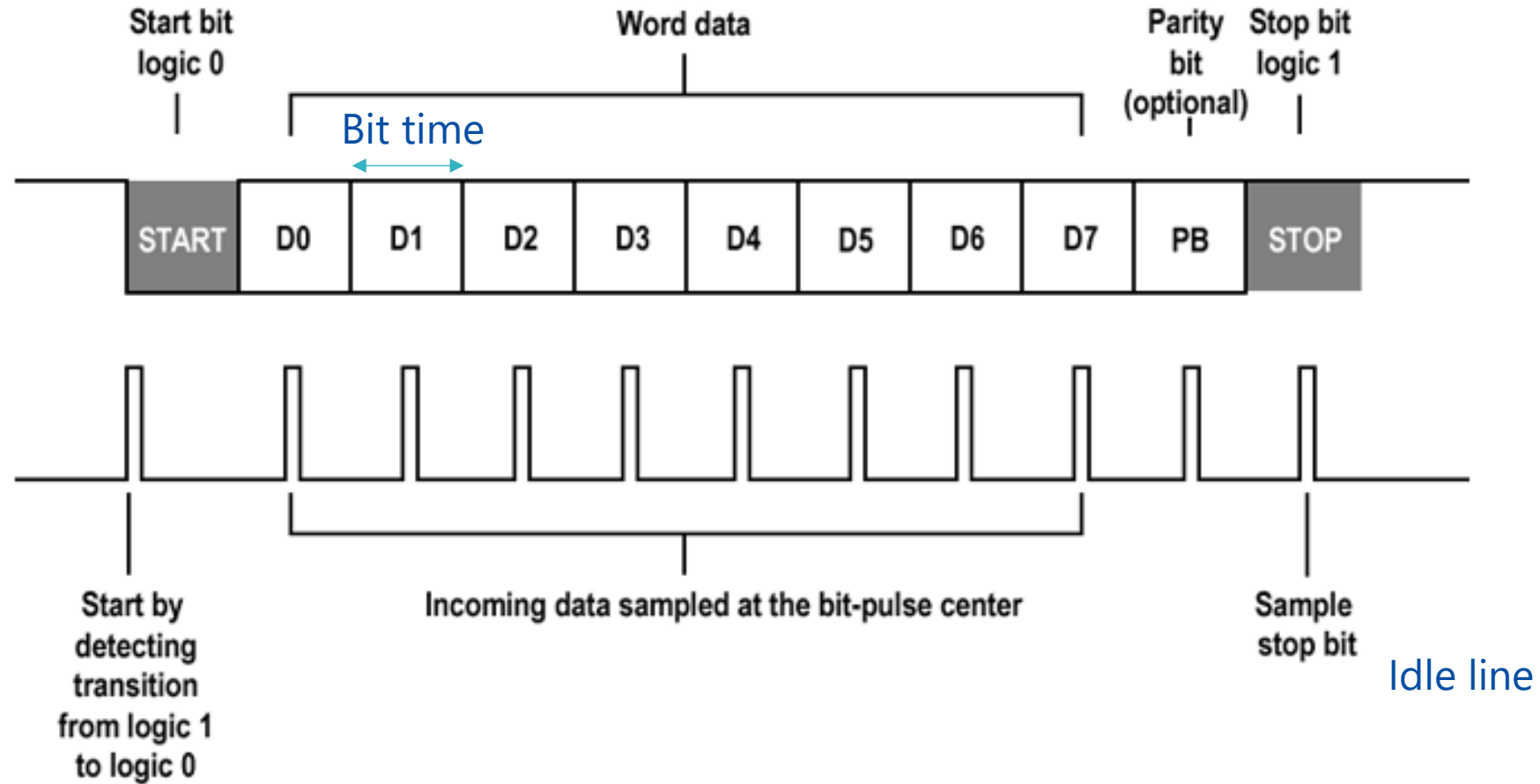
2.4. Parity bit(optional)

- ❖ Used for error detection
- ❖ Even parity: number of 1's must be even
- ❖ Odd parity: number of 1's must be odd

Example: ASCII 'H'(0x48= 1 0 0 1 0 0 0) = two 1's



2.5. UART Timing



Idle line

The baud rate is the rate at which information is transferred in a communication channel. In the serial port context, "9600 baud" means that the serial port is capable of transferring a maximum of 9600 bits per second.

2.6. UART Data throughput

Data throughput Example

- + Assume 19200 baud rate , 8 data bits, no parity, 1 stop bit
 - Baud rate 19200 $\rightarrow$ 19.2kbps
 - 1 start bit + 8 data bits + 1 stop bit = 10 bits
 - It takes 10 bits to send 8 bits of data.
 - $19.2 \text{ kbps} * 8/10 = 15.36 \text{ kbps}$.
 - How many KB (kilobytes) per second:
 $= 15.36 * 1\text{KB} / 8192 \text{ bits} = 1.875 \text{ KB/s}$

3. Universal synchronous asynchronous receiver transmitter (USART)

USART main features:

- Full duplex, asynchronous communications
- NRZ standard format (Mark/Space)
- Configurable oversampling method by 16 or by 8 to give flexibility between speed and clock tolerance
- Fractional baud rate generator systems
- Programmable data word length (8 or 9 bits)
- Configurable stop bits - support for 1 or 2 stop bits
- Single-wire half-duplex communication
- Configurable multibuffer communication using DMA
- Separate enable bits for transmitter and receiver

USART/UART counts:

- 4 USARTs: USART1, USART2, USART3, USART6
- 4 UARTs : UART4, UART5, UART7, UART8

3.1. Setting Tx/Rx pin

Set alternate function mode pin:

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..I/J/K)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

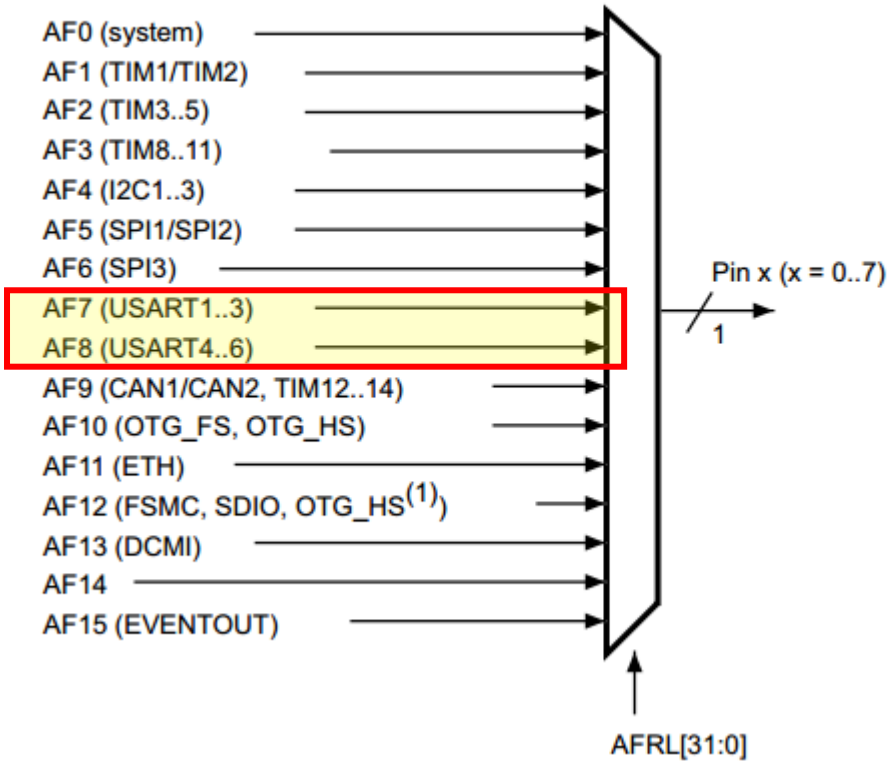
10: Alternate function mode

11: Analog mode

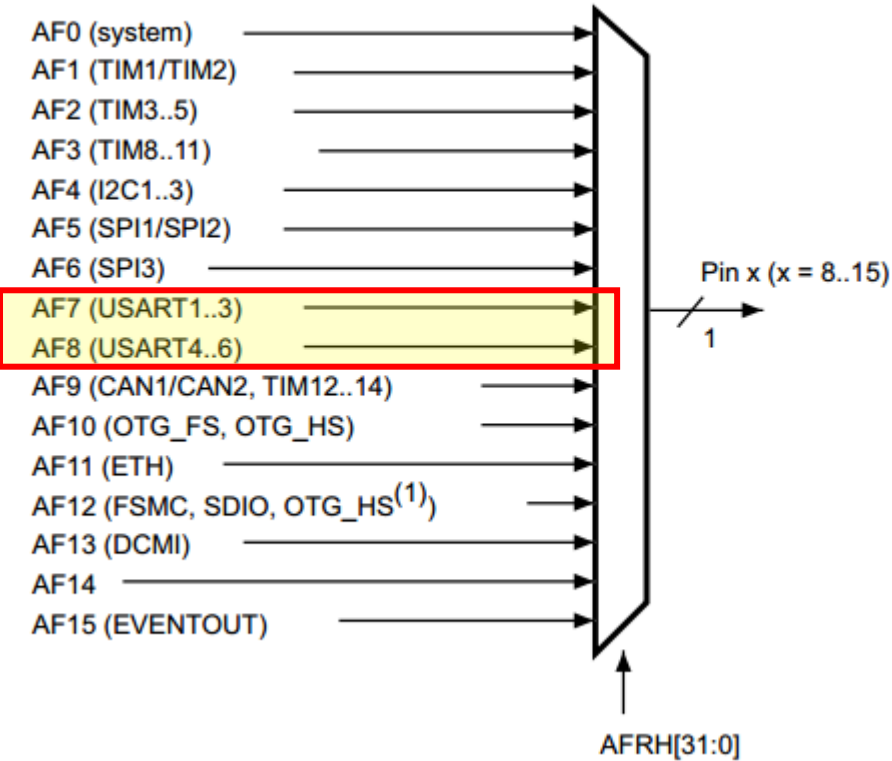
3.1. Setting Tx/Rx pin

Selecting an alternate function for pin

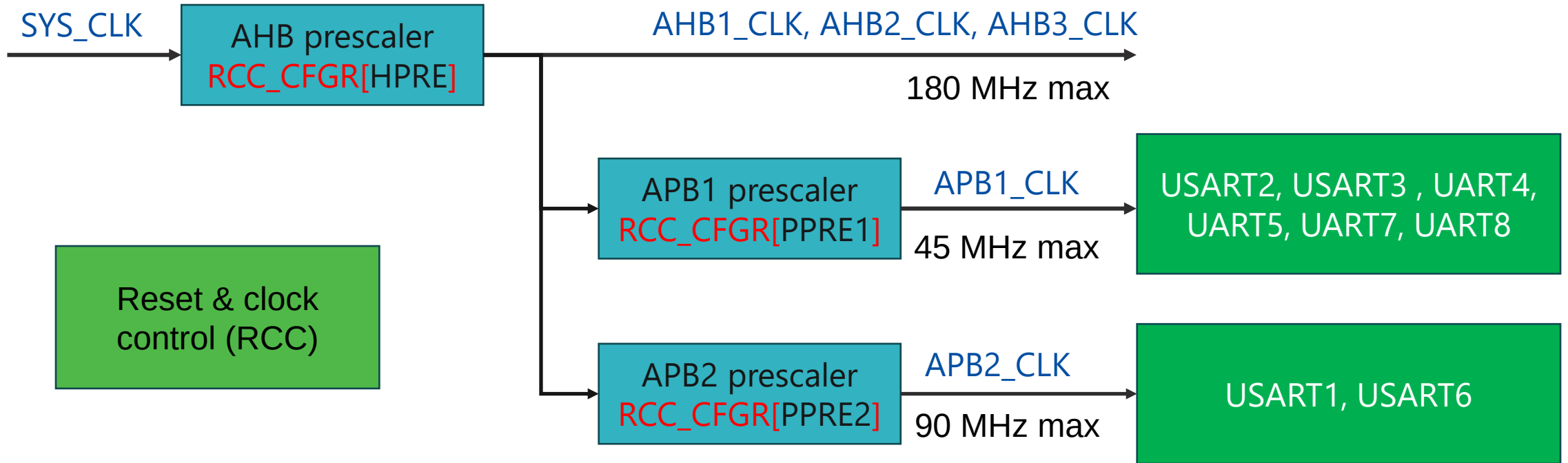
For pins 0 to 7, the GPIOx_AFRL[31:0] register selects the dedicated alternate function



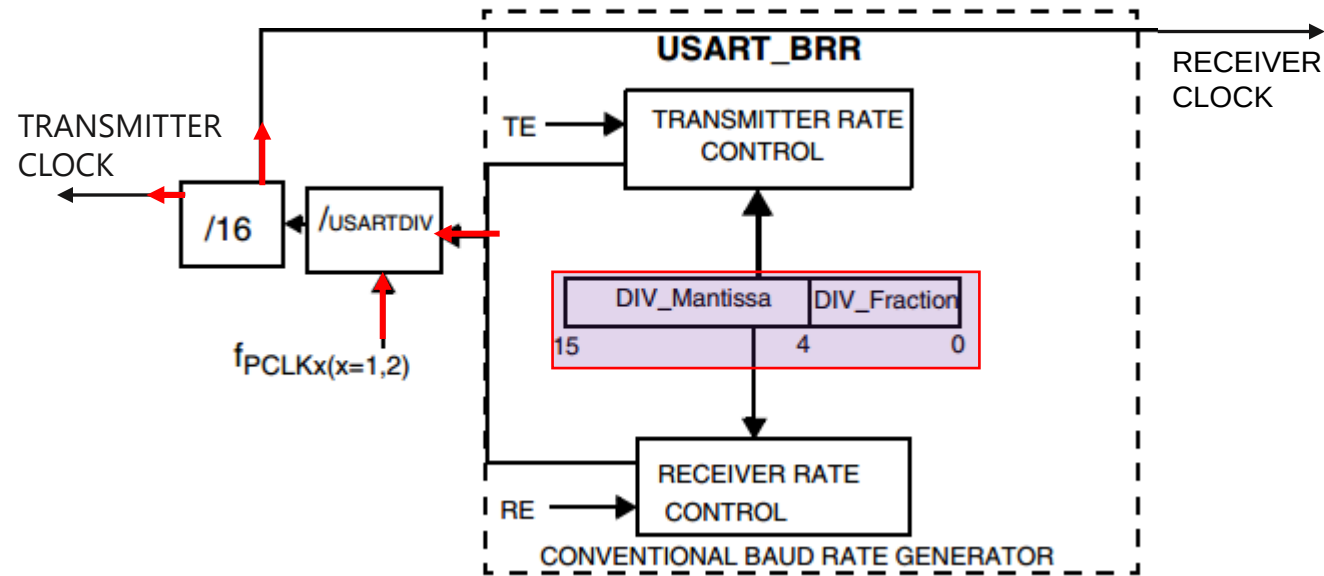
For pins 8 to 15, the GPIOx_AFRH[31:0] register selects the dedicated alternate function



3.2. Setting Clock



3.3. Baud rate generator



1. Select the Oversampling mode: oversampling by 16 or 8
2. Select baud rate: 4800, 9600, 14400, 19200, 38400, 56000, 57600, 115000
3. Calculate Mantissa and Fraction

$$\text{Tx/Rx baud} = \frac{f_{CK}}{8 \times (2 - \text{OVER8}) \times \text{USARTDIV}}$$

$$\text{USARTDIV} = \text{DIV_Mantissa} + (\text{DIV_Fraction} / \text{OVER8})$$

3.3. Baud rate generator

Baud rate register (USART_BRR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value

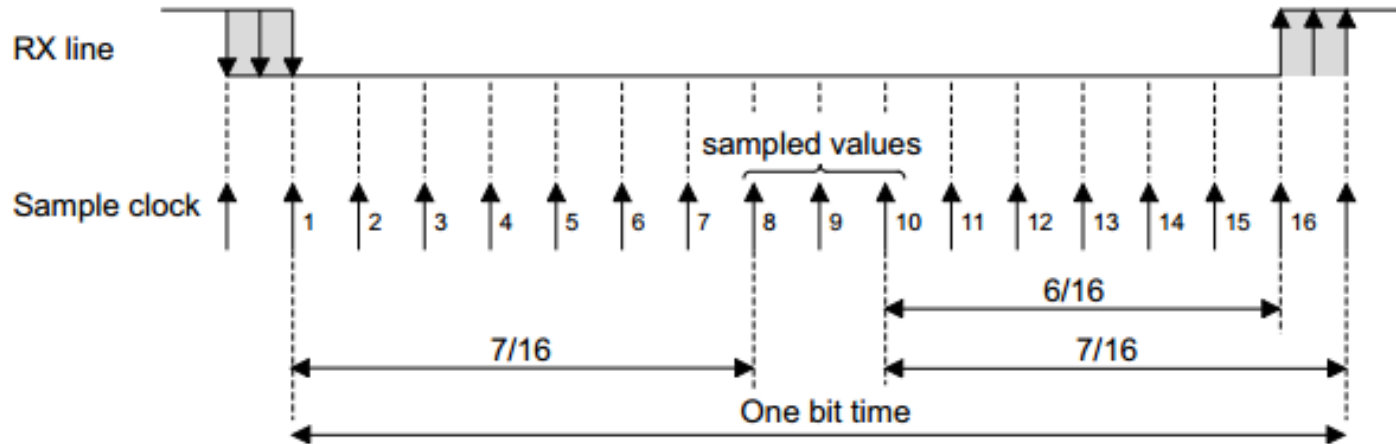
Bits 15:4 **DIV_Mantissa[11:0]**: mantissa of USARTDIV

These 12 bits define the mantissa of the USART Divider (USARTDIV)

Bits 3:0 **DIV_Fraction[3:0]**: fraction of USARTDIV

These 4 bits define the fraction of the USART Divider (USARTDIV). When OVER8=1, the DIV_Fraction3 bit is not considered and must be kept cleared.

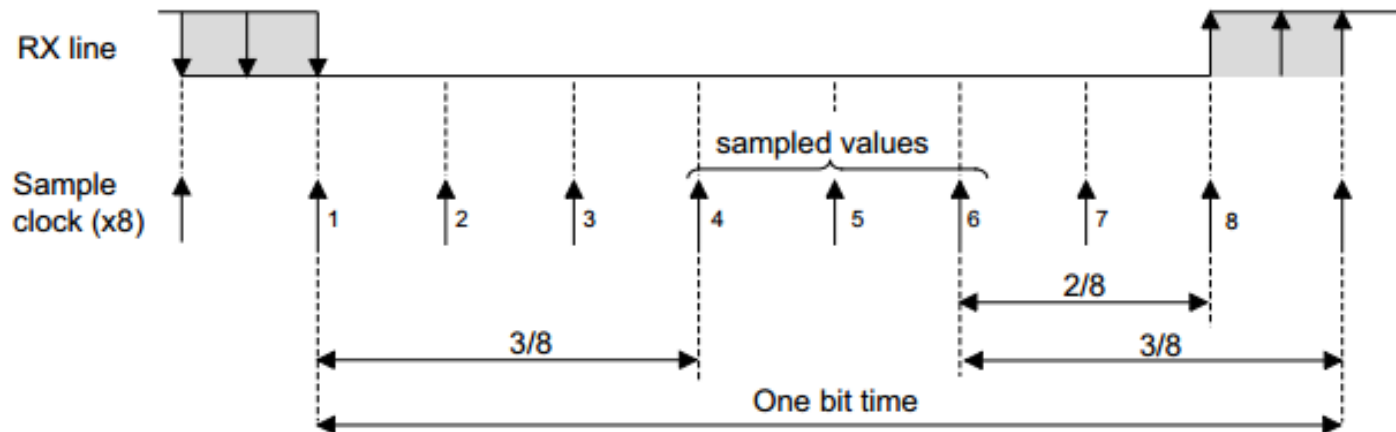
3.4. Oversampling



Data sampling when oversampling by 16

Table 133. Noise detection from sampled data

Sampled value	NE status	Received bit value
000	0	0
001	1	0
010	1	0
011	1	1
100	1	0
101	1	1
110	1	1
111	0	1



Data sampling when oversampling by 8

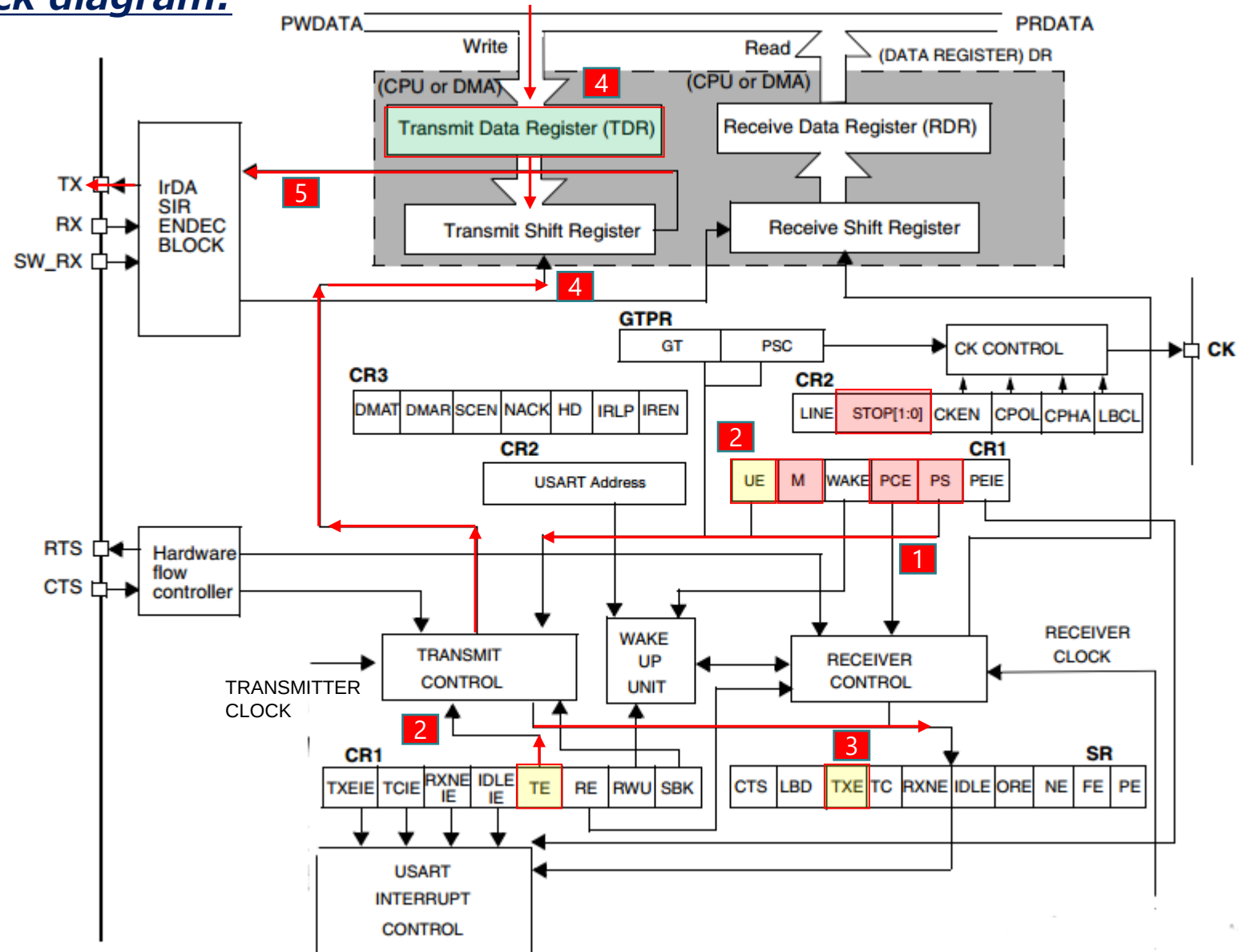
Control register 1 (USART_CR1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 15 **OVER8**: Oversampling mode
0: oversampling by 16
1: oversampling by 8

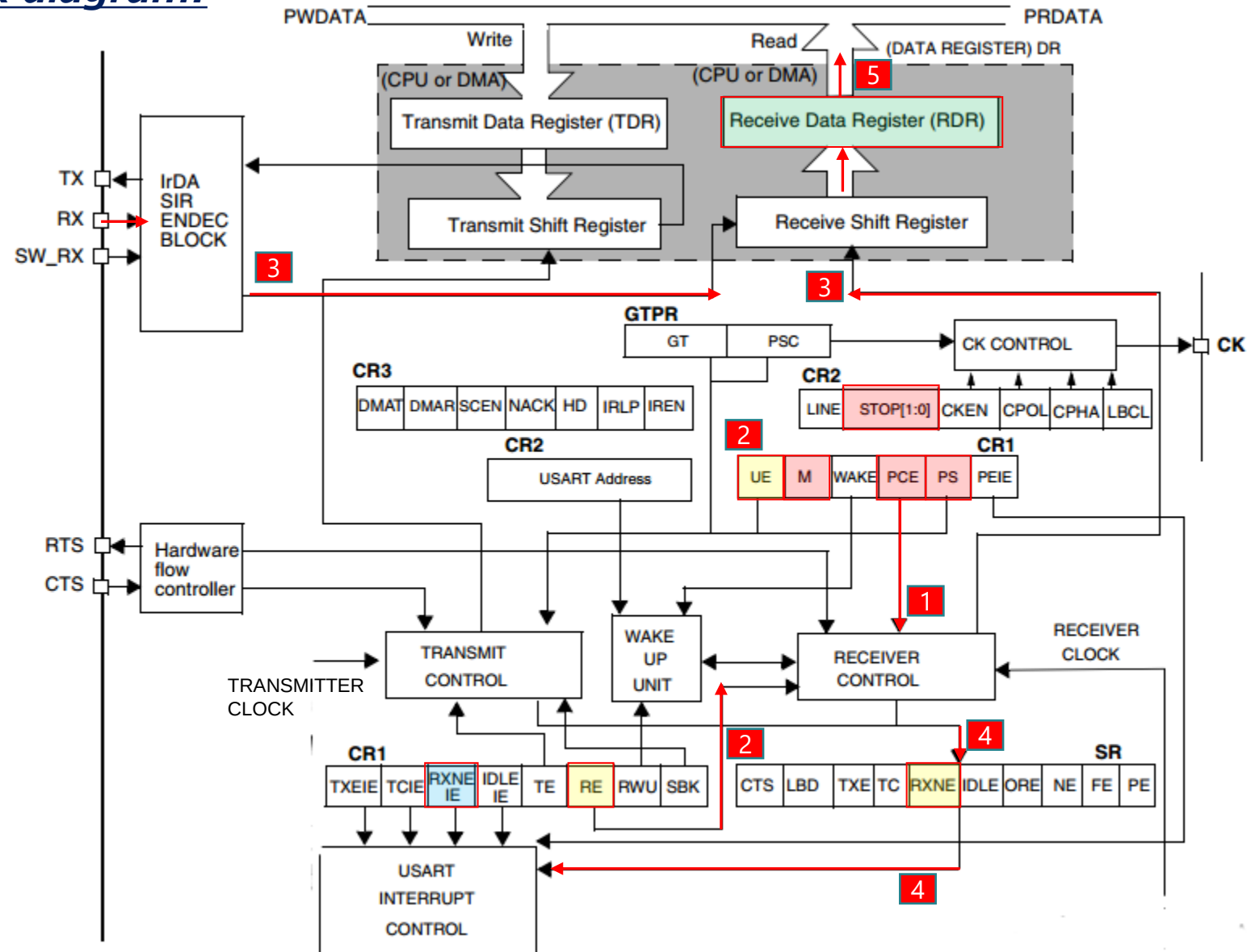
3.5. Block diagram

USART transmitter block diagram:



3.5. Block diagram

USART receiver block diagram:



3.6. USART register map

USART register map and reset values:

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x00	USART_SR	Reserved																						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE		
	Reset value																							0	0	1	1	0	0	0	0	0	0		
0x04	USART_DR	Reserved																						DR[8:0]											
	Reset value																							0	0	0	0	0	0	0	0	0			
0x08	USART_BRR	Reserved														DIV_Mantissa[15:4]								DIV_Fraction[3:0]											
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0							
0x0C	USART_CR1	Reserved														OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK				
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					
0x10	USART_CR2	Reserved														LINEN	STOP[1:0]	CLKEN	CPOL	CPHA	LBCL	Reserved	LBDIE	LBDL	Reserved	ADD[3:0]									
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0							
0x14	USART_CR3	Reserved																						ONEBIT	CTSIE	CTSE	RTSE	DMAT	DMAR	SCEN	NACK	HDSEL	IRLP	IREN	EIE
	Reset value																							0	0	0	0	0	0	0	0	0	0	0	0
0x18	USART_GTPR	Reserved														GT[7:0]						PSC[7:0]													
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0							

3.6.1. Assignment 1

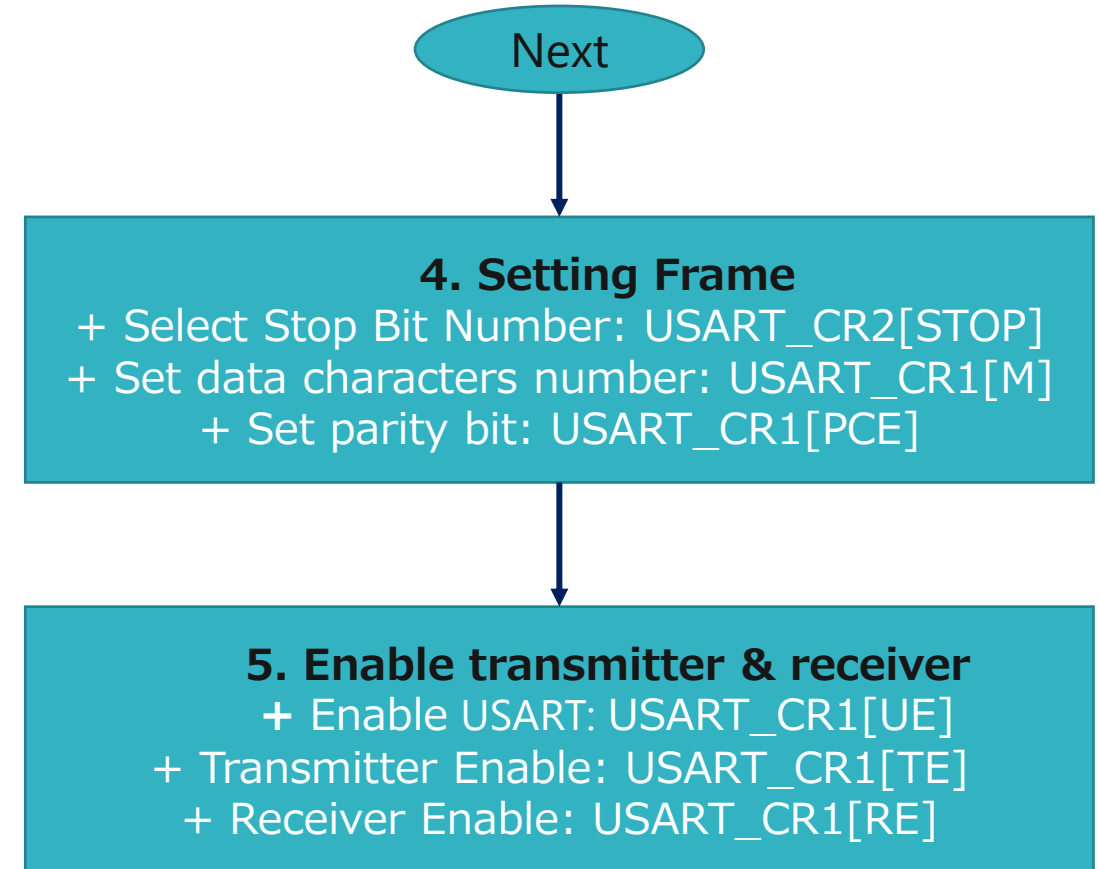
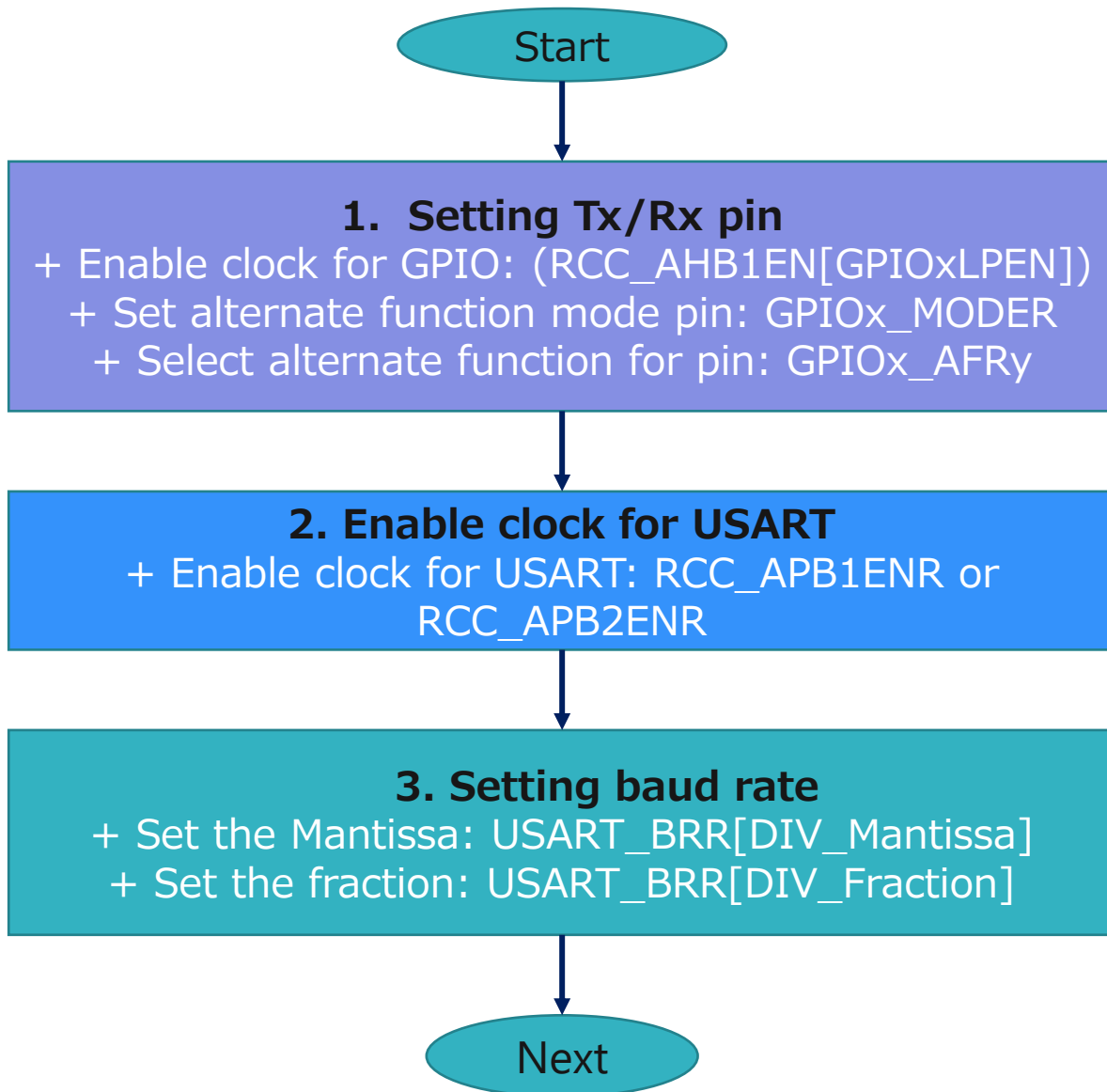
Write a define struct to manage USART register map

Ex:

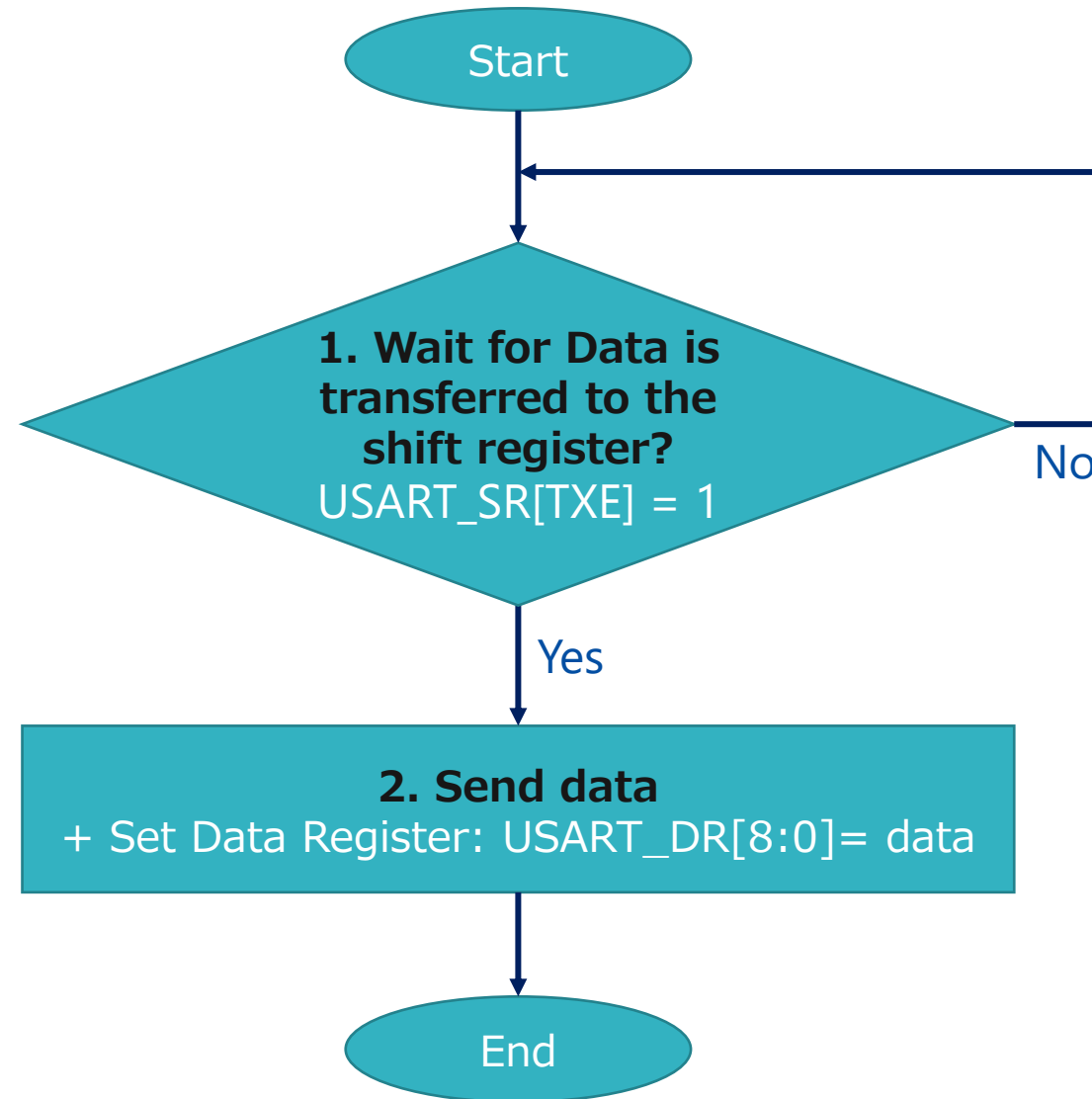
```
typedef struct {  
    register  
    .....  
} USART_Type;
```

```
#define USART1_BASE_ADDRESS (0x40011000 u)  
#define USART1 ((LPUART_Type*) USART1_BASE_ADDRESS )
```

3.7. Configuration for USART



3.8. Configuration for USART transmit single Char

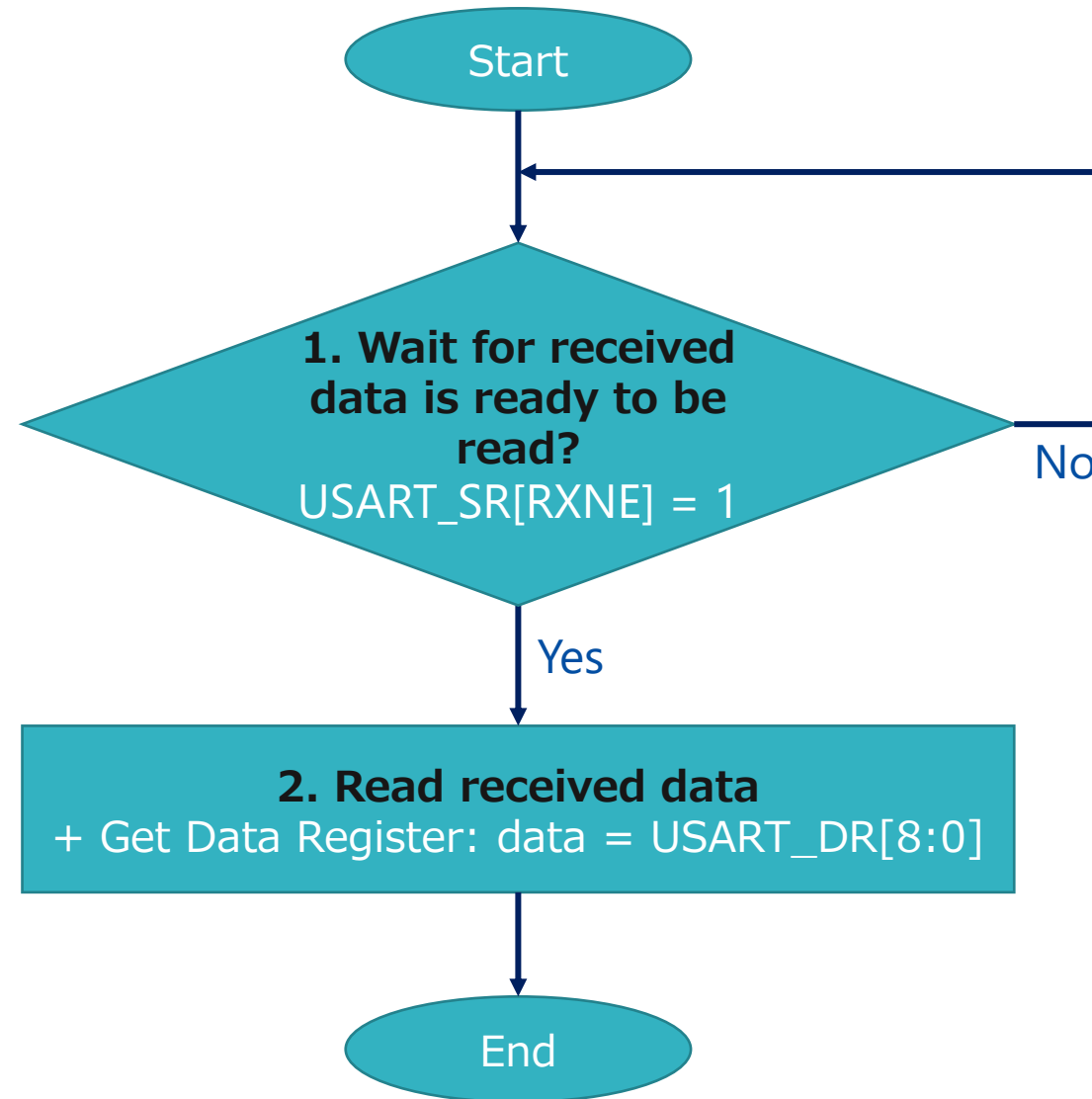


3.8.1. Assignment 2

Write a program to **Transmit** single char via USART

- Baud rate: 9600
- 1 stop bit
- 8 bit format
- no parity
- Send char: 'h'

3.9. Configuration for USART receive single Char



3.9.1. Assignment 3

Write a program to **Receive** single char via USART

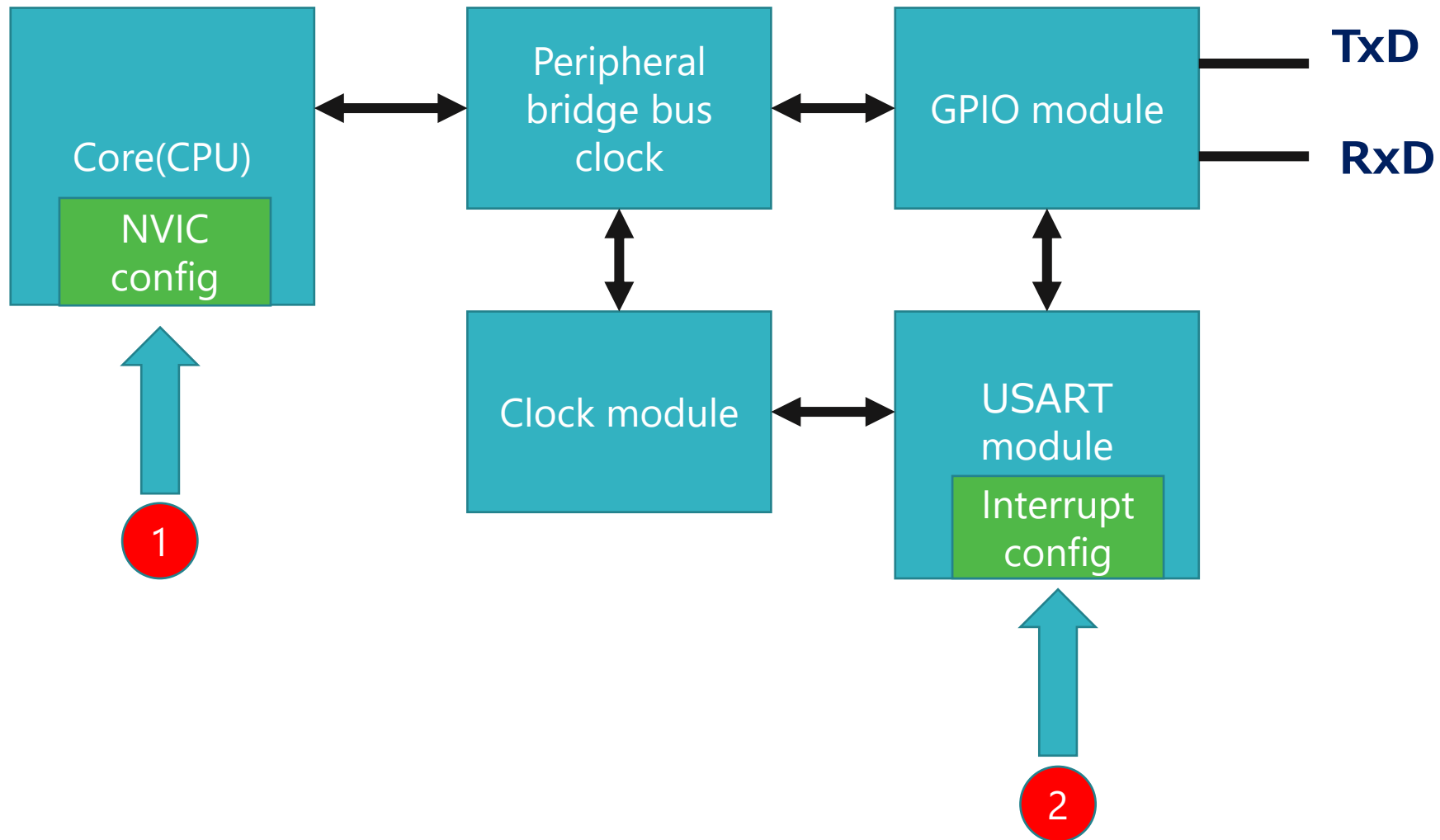
- Baud rate: 9600
- 1 stop bit
- 8 bit format
- no parity
- if single char received is '1' then led on
else single char received is '0' then led off

3.10. Assignment 4

Write a program to **Transmit** a string via USART

- Baud rate: 9600
- 1 stop bit
- 8 bit format
- no parity
- Send string: "Hello world/r/n"

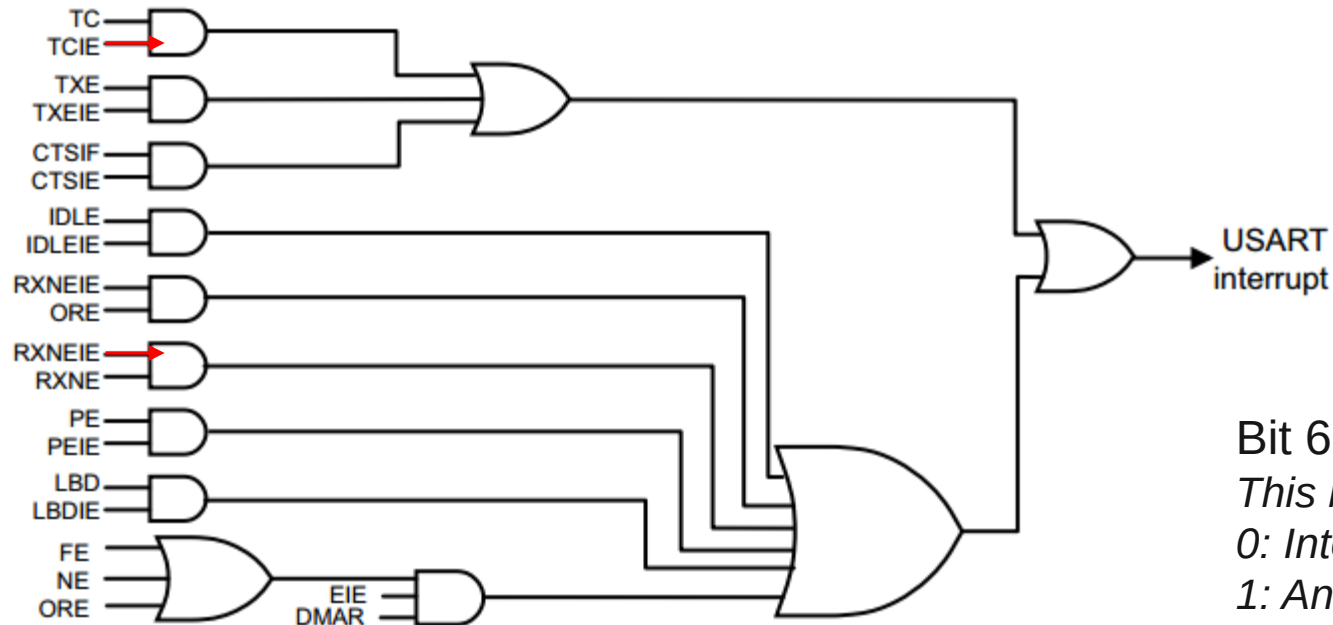
4. LPUART Interrupt



4. USART Interrupt

Control register 1 (USART_CR1): Enable Receiver, Transmission Complete Interrupt

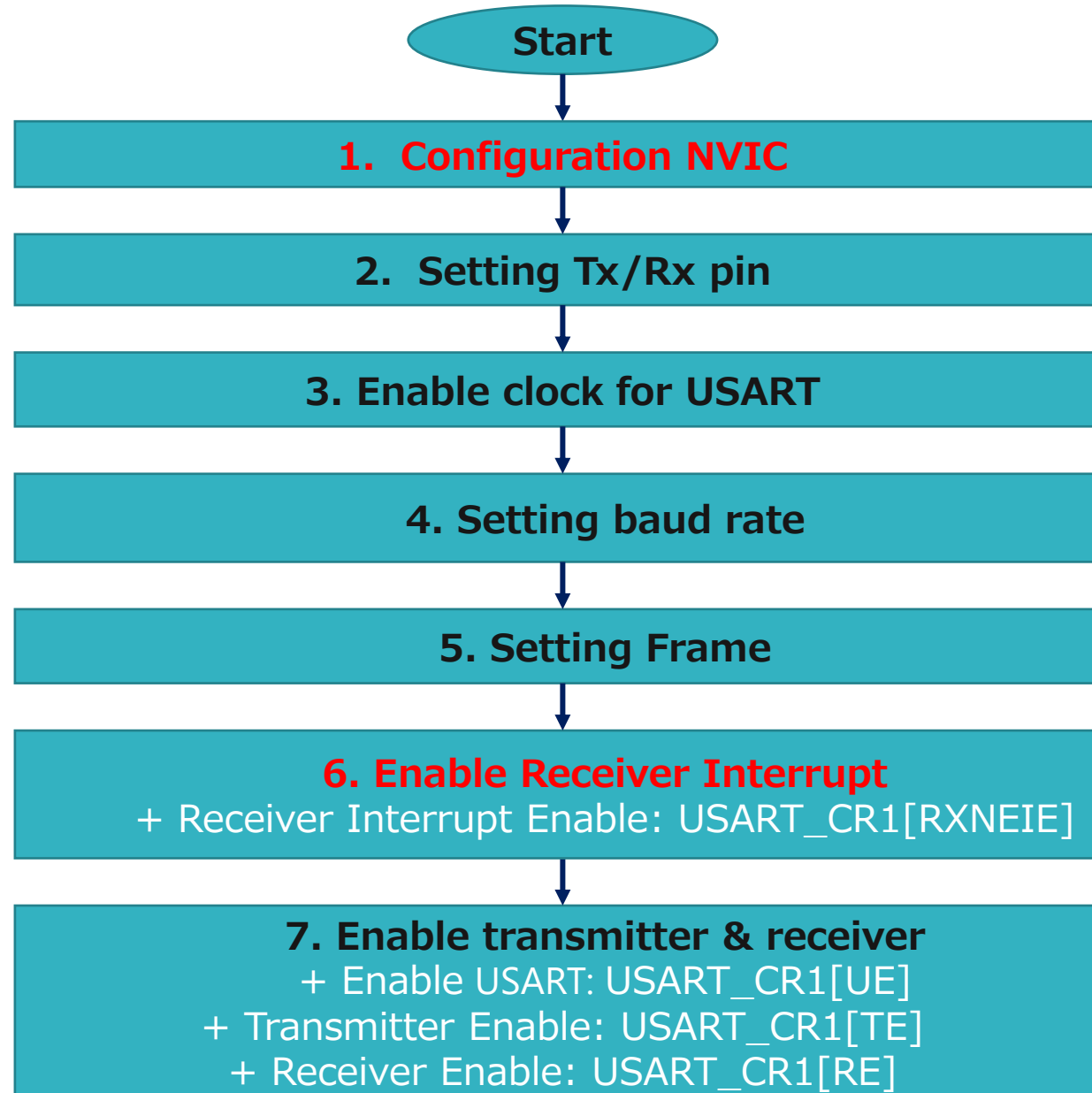
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



Bit 5 RXNEIE: RXNE interrupt enable
This bit is set and cleared by software.
0: Interrupt is inhibited
1: An USART interrupt is generated whenever ORE=1 or RXNE=1 in the USART_SR register

Bit 6 TCIE: Transmission complete interrupt enable
This bit is set and cleared by software.
0: Interrupt is inhibited
1: An USART interrupt is generated whenever TC=1 in the USART_SR register

4.1. USART Receiver Interrupt



4.2. Assignment 5

Write a program USART **Receiver interrupt**

- Baud rate: 9600
- 1 stop bit
- 8 bit format
- no parity
- if single char received is '**1**' then led on
else single char received is '**0**' then led off

4.3. Assignment 6

Write a program USART **Receiver interrupt**

- Baud rate: 9600
- 1 stop bit
- 8 bit format
- no parity
- if a string received is "[LEDON]" then led on
else a string received is "[LEDOFF]" then led off

A nighttime cityscape featuring a prominent skyscraper with a spire, illuminated by warm lights. The building is framed by a large, semi-transparent, stylized letter 'R' in a light purple/pink hue. Other buildings and a body of water are visible in the background under a dark sky with some clouds.

Thank you